

CM/TD 2 — UE3 : BIG DATA - SCALA



ING 3 : Option Intelligence Artificielle – 2026/2027

Rakib SHEIKH ^{1,2}

rakib.sheikh@cyu.fr

¹ Thales Romania SRL

² CY Cergy Paris Université - CY-Tech

01

Le langage Scala en Big Data (J2)





Plan de la journée

- Cours Magistrale pour débiter la journée
- TP2 partie 1 : Application direct des points vu en CM
- **Pause déjeuner (12h00 à 13h30)**
- TP2 partie 2 : Handson Scala
- **Fin de journée à 17h00**



Introduction

Utilisé principalement par les Data Engineers pour réaliser les traitements de données.

Liste des librairies usuelle :

- Apache Spark (Traitement Batch)
- Apache Kafka (Traitement Temps réel)
- Apache Spark Mlib (équivalent scikit-learn)
- Hadoop HDFS

But de Scala appliqué au Big Data :

- **Éliminer les codes répétitifs liée à Java !** Autrement dit : Pas de Boilerplates.

Le cours est basé de la documentation et du guide officiel de Scala :

<https://docs.scala-lang.org/tutorials/scala-for-java-programmers.html>



Le langage Scala

- Si vous avez bien suivi les cours de Java, surtout sur les Conteneurs, Streams et Lambda, vous savez déjà coder sous Scala !
- Si vous avez compris le paradigme fonctionnelle : Bravo, vous pouvez en abuser ici !
- Reprend la quasi-totalité de la syntaxe Java.
- Simplification au max du code Java.

Point d'attention:

- La dernière version de Scala est la 3.5.2
- Cependant nous utiliserons la 2.13.15

Pourquoi ?

- Les librairies sous Apache n'ont pas migré le codebase vers la version 3.
- A l'inverse de Java, Scala n'assure pas la rétro-compatibilité à chaque version majeur.





Notre première classe.

Pour créer une classe Hello sous Scala :

```
1 object Hello extends App {  
2     println("Hello")  
3 }
```

scala

- Ici, nous remarquons que nous avons la même syntaxe de `println` sous Java 23. (Non applicable à Java 25 dont l'API final est `IO.println()`)
- En effet, avant Java 23, nous devions écrire `System.out.println()`
- L'`extends App` est pratique pour lancer un fichier unique Scala, mais non recommandé.
- Le mot clé **object** est une classe particulière : **Il y a qu'une seule instance possible.**



Ajoutons une méthode main

```
1 object Hello {  
2     def main(args: Array[String]): Unit = {  
3         println("Hello")  
4     }  
5 }
```

scala

- Nous définissons une méthode avec une `def`
- Les arguments et le type sont inversés.
- Le `=` désigne une affectation de valeur.
- Le `Unit` désigne le type de retour.

Remarque : Nous perdons `extends App` car nous avons une fonction Main.



Pour compiler et exécuter un programme

```
1 scalac Hello.scala # pour compiler
```

bash

```
2 scala Hello # pour executer
```



Maintenant, complexifions le code avec du Java.

Rédigeons un programme affichant l'heure pour la République de Corée.

```
1  import java.time.format.{DateTimeFormatter, FormatStyle}
2  import java.time.LocalDate
3  import java.util.Locale._
4  object ReplubicOfKoreaDate {
5      def main(args: Array[String]): Unit = {
6          val now = LocalDate.now
7          val df = DateTimeFormatter
8              .ofLocalizedDate(FormatStyle.LONG)
9              .withLocale(KOREA)
10         println(df.format(now))
11     }
12 }
```

Scala



Une classe normale sous Scala version naïf

Les classes fonctionnent de la même manière que Java.

```
1  class Cat {
2      var name: String = _
3      var age: Int = _
4      var species: String = _
5      def setAttributes(name: String, age: Int): Unit = {
6          this.name = name
7          this.age = age
8      }
9      def speak(): Unit = {
10         println(s"Meow, I'm $name and I'm $age years old.")
11     }
12 }
```

scala



Une classe normale sous Scala version naïf

```
13 val myCat = new Cat
14 myCat.setAttributes("Whiskers", 3)
15 myCat.speak() // Output: Meow, I'm Whiskers and I'm 3 years old.
```

- Ici, on ne peut pas dès instantiation de la classe, attribuer des valeurs.
- On est dans l'obligation de définir un constructeur pour pouvoir attribuer des valeurs.
- La méthode naïf nous fait **perdre du temps**.



Exemple optimisé (utilisation des factory)

```
1 class Cat(name: String, age: Int) {  
2     var species: String = _  
3     def speak(): Unit = {  
4         println(s"Meow, I'm $name and I'm $age years old.")  
5     }  
6 }  
7  
8 val myCat = new Cat("Whiskers", 3) // Create an instance of Cat  
9 myCat.speak() // Output: Meow, I'm Whiskers and I'm 3 years old.
```

scala

- Les attributs initialisé par constructeur directement dans le nom de la classe (comme une record).
- Autorisation d'avoir plusieurs constructeurs.

Object

Object Une Object est **une instance** de leur propre définition. Ce sont des singletons de leur propre classes.

```
1 object IdFactory {  
2     private var counter = 0  
3     def create(): Int = {  
4         counter += 1  
5         counter  
6     }  
7 }
```

scala

```
1 val newId: Int =  
  IdFactory.create()  
2 println(newId) // 1  
3 val newerId: Int =  
  IdFactory.create()  
4 println(newerId) // 2
```

scala



Remarque importante : Classe vs Object

But	class	object
But	Modèle pour créer des instances	Objet singleton (une seule instance)
Instanciation	Peut être instanciée avec <code>new</code>	Ne peut pas être instancié (toujours unique)
État	Peut avoir un état mutable ou immuable (champs)	Pas d'état mutable (champs immuables)
Utilisation	Représente des entités avec état et comportement	Utilisé pour des méthodes utilitaires, des constantes ou des objets compagnons
Héritage	Peut hériter d'autres classes	Peut étendre des classes, des traits (mais une seule instance)
Méthode <code>main</code>	Ne peut pas définir la méthode <code>main</code>	Peut définir une méthode <code>main</code> (point d'entrée pour les programmes)
Compagnon	Peut avoir un objet compagnon associé	Souvent utilisé comme objet compagnon pour les classes



Une record du Java 16 sous Scala

Pour rédiger une record sous Java

```
1 record Personne(String prenom, String nom) {}
```

java

Pour rédiger une record sous Scala appelé **case classe**

```
1 case class Personne(prenom: String, nom: String)
```

scala

Remarque :

- Nous inversons la déclaration d'une variable et de son type dans les méthodes !
- Les instances de case class sont comparé par leur leur et non par référence

Rappel:

- Une record/case class est une classe dont le constructeur, toString, hashCode, equals et les getteurs sont automatiquement généré par le compilateur à la compilation.



Une record du Java 16 sous Scala

```
1 case class Personne(prenom: String, nom: String) scala
2 object Main {
3     def main(args: Array[String]): Unit = {
4         val personnel: Personne = new Personne("Rakib", "SHEIKH")
5         println(personnel)
6     }
7 }
```

- Pour instancier une classe, comme en Java, nous utilisons le mot clé `new`
- Remarque : Nous n'avons pas appelé `toString` à `personnel`
 - En effet, comme en Java, le compilateur va automatiquement appeler `toString()`



La base : Object

Le langage Scala est entièrement orienté objet, ce qu'il veut dire que littéralement tout est un **Object**. **Y compris les nombres**.

- Par comparaison au Java, tout objet héritent de `java.Object`.
- Sous Scala, ca sera `scala.AnyRef`.

Ce code :

```
1 1 + 2 * 3 / x
```

scala

est équivalent à ce code :

```
1 1.+(2.*(3)./(x))
```

scala

Nous pouvons déduire qu'à travers cette méthode, `+` est une méthode !



Les méthodes

Les méthodes ne changent pas par rapport à Java : Elle sont reconnaissable grâce à un `def` devant le nom.

```
1 def add(x: Int, y: Int): Int = x + y
2 println(add(1, 2)) // 3
```

scala

Avec de paramètres multiples

```
1 def addThenMultiply(x: Int, y: Int)(multiplier: Int): Int = {
2     (x + y) * multiplier
3 }
4 println(addThenMultiply(1, 2)(3)) // 9
```

scala

Remarque: La dernière ligne est toujours la valeur de retour, on utilise rarement `return`.



Les fonctions

Les fonctions sont des expressions qui ont des paramètres et prennent des arguments.

Exemple avec une fonction nommée

```
1 val addOne = (x: Int) => x + 1 // Avec 1 Paramètre
```

Scala

```
2 println(addOne(1)) // 2
```

```
3
```

```
4 val add = (x: Int, y: Int) => x + y // Avec 2 Paramètres
```

```
5 println(add(1, 2)) // 3
```

```
6
```

```
7 val getTheAnswer = () => 42 // Aucun Paramètres
```

```
8 println(getTheAnswer()) // 42
```



Les fonctions anonymes

Les fonctions anonyme sous Scala fonctionnent de la même manière que les Lambdas sous Java mais à une différence près :

- Nous n'avons pas besoin d'avoir une interface fonctionnel

Vous pouvez définir une fonction anonyme (c'est-à-dire une fonction qui n'a pas de nom) qui renvoie un entier donné plus un :

```
1 (x: Int) => x + 1
```

scala



Avant, avec Java 25

Java

```
1  public interface Printable {
2      String print(String s);
3  }
4
5  // Fichier Main.Java sous Java 25, ici la classe Main est inféré
   directement
6  void main() {
7      Printable lambdaCat = (s) -> IO.println("Meow" + s);
8      printThing(lambdaCat);
9  }
10 static void printThing(Printable thing) {
11     thing.print("!");
12 }
```



Après, avec Scala

```
1  object Main extends App {
2      // Lambda that prints to the console (Unit return type)
3      val lambdaCat: (String => Nothing) = (s: String) => {
4          println("Meow" + s)
5      }
6      // Method to call print on the lambda
7      def printThing(thing: String => Unit): Unit = {
8          thing("!") // Calls the lambda with "!" as input
9      }
10     // Execute the function
11     printThing(lambdaCat)
12 }
```

scala

En gros ce qu'il faut retenir: `() => {Mon code}`



Methodes VS Functions

⊘ | Danger

- Les méthodes sont définies avec un `def`
- Les fonctions sont définies avec une `val`

C'est une nouveauté par rapport à Java.

- Il faut voir les trait comme une interface qui contient du code.

Quand une classe hérite d'une trait, elle va implémenter la traite **ET** importer l'ensemble des code qui y sont.



Trait (Exemple)

Définissons une trait `SoundMaker` qui possède une méthode abstraite `makeSound` et une autre non-abstraite `description`

```
1 trait SoundMaker {  
2     def makeSound(): Unit  
3  
4     def description: Unit = {  
5         println("I can make a sound!")  
6     }  
7 }
```

scala

Implémentons maintenant deux classes

- Dog
- Cat



Trait (Exemple)

```
1  class Dog extends SoundMaker {
2      def makeSound(): Unit = { // Implementing the abstract method
3          println("Woof!")
4      }
5  }
6
7  class Cat extends SoundMaker {
8      def makeSound(): Unit = { // Implementing the abstract method
9          println("Meow!")
10     }
11 }
```

scala

Ici les deux classes sont différentes, mais implémentent le trait `SoundMaker`. Nous implémentons alors `makeSound` !



Trait (Exemple)

Et voici ce que ça donne à travers un main:

```
1  object Hello extends App {
2      // Usage
3      val myDog = new Dog
4      myDog.makeSound()    // Output: Woof!
5      myDog.describe()    // Output: I can make a sound!
6
7      val myCat = new Cat
8      myCat.makeSound()    // Output: Meow!
9      myCat.describe()    // Output: I can make a sound!
10 }
```

scala



Une variante : Interfaces scellées sealed

Une sealed trait est tout simplement une restriction sur la manière l'héritage fonctionne :

- L'ensemble des classe qui héritent du trait **devront être dans le même fichier !**

```
1 sealed trait Shape
2 case class Circle(radius: Double) extends Shape
3 case class Rectangle(width: Double, height: Double) extends Shape
4 case class Square(side: Double) extends Shape
```

scala



Patterns Matching

C'est l'une des structure de contrôle les plus puissant !

- L'avantage est que l'on peut rédiger plusieurs cas possible, sur les valeur voir **même sur le type de l'objet !**

Reprenons l'exemple d'une `sealed trait`:

- Nous allons définir une méthode qui va calculer l'aire d'une géométrie en fonction de son type (Cercle, Rectangle ou Carré)

```
1 def calculateArea(shape: Shape): Double =
2   shape match {
3     case Circle(radius) => Math.PI * radius * radius
4     case Rectangle(width, height) => width * height
5     case Square(side) => side * side
6   }
7 }
```

scala



Patterns Matching

Remarque :

- Avez-vous remarqué qu'il y a pas de cas de base ?

Nous sommes dans une interface scellées : à partir du moment où tous les cas sont couverts, il n'y a pas besoin du cas de base.

En revanche, en dehors des traits, il faut absolument un cas de base :

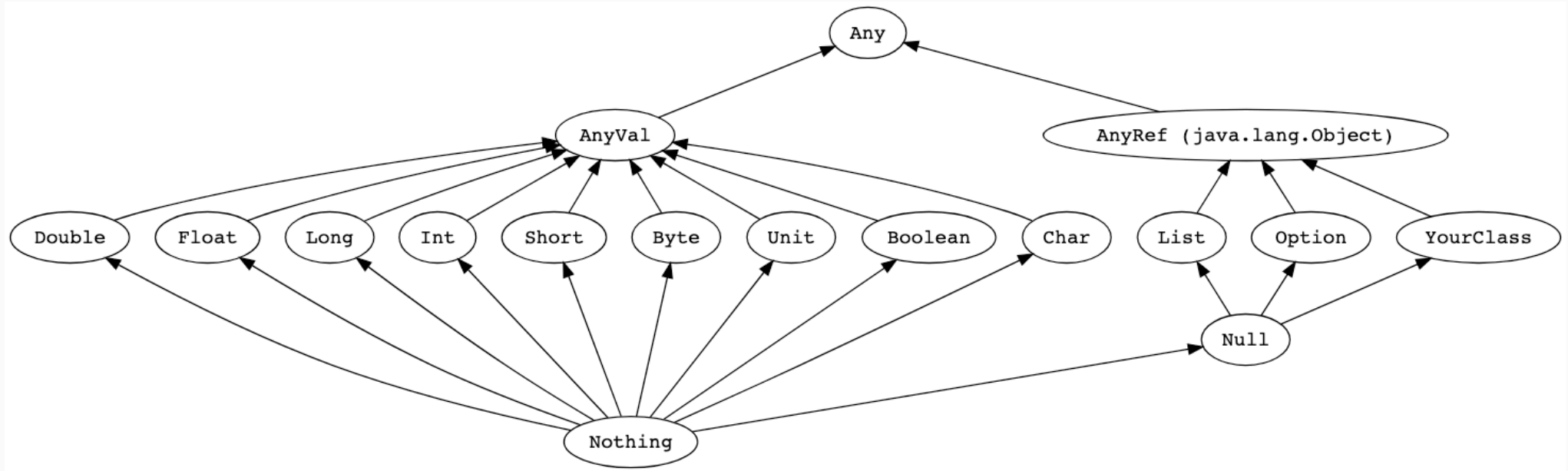
```
1 def calculateArea(shape: Shape): Double = shape match {  
2   case Circle(radius) => Math.PI * radius * radius  
3   case Rectangle(width, height) => width * height  
4   case _ => 0.0 // Default case for unknown shapes  
5 }
```

scala

Cette exemple suppose que nous sommes pas dans un trait, et qu'un cas de base est nécessaire.



Types spécifiques



- Ici on voit bien la référence `Any` être au sommet des types
- Nous remarquons deux types nouveau **Nothing** et **Null**



Nothing

i | Définition : **Nothing**

Un sous-type de tous les types, également appelé type inférieur. Il n'existe aucune valeur de type `Nothing`.

Il est notamment utilisé comme type de retour pour les expressions qui ne produisent normalement aucune valeur, par exemple:

- *une exception lancée*
- *un arrêt du programme*
- *une boucle infinie.*

```
1 def fail(message: String):  
  Nothing =  
2    throw new  
    RuntimeException(message)  
3  
4 def infiniteLoop(): Nothing =  
5   infiniteLoop()
```

scala

i | Définition : **Null**

C'est un sous-type de tous les types de référence (c'est-à-dire des sous-types de `AnyRef`). Il possède une seule valeur identifiée par le mot-clé `null`.

Null existe principalement pour l'interopérabilité avec les autres langages de la JVM. En Scala, il est généralement préférable d'utiliser `Option` plutôt que `null`.

None C'est ce qu'il faut utiliser à la place de Null ! Et donc si on est pas sûr d'avoir une réponse, on utilise **`Some(valeur)`** qui va renvoyer soit sa valeur, soit `None`. Ce qui implique **`Option[A]`**

Exemple avec une méthode

```
1 def findUserId(id: Int): Option[String] = {  
2   // Pretend we're searching in a database  
3   if (id == 1) Some("Alice")  
4   else None
```

scala

Application directe via un match

```
1 findUserId(1) match  
2   case Some(name) => println(s"Utilisateur trouvé : $name")  
3   case None       => println("Utilisateur introuvable")
```

scala



Types génériques

La syntaxe d'une classe générique change par rapport à Java. Mais sa fonction reste inchangé.

- Nous les rédigeons sous la forme suivante :

```
1  class Fruit
2  class Apple extends Fruit
3  class Banana extends Fruit
4
5  val stack = new Stack[Fruit]
6  val apple = new Apple
7  val banana = new Banana
8
9  stack.push(apple)
10 stack.push(banana)
```

scala



Higher Order Function

Attention : très utilisé chez les Data Engineers

Les fonctions d'ordre supérieur prennent d'autres fonctions en paramètre ou renvoient une fonction comme résultat.

Nous utilisons l'expression « fonction d'ordre supérieur » à la fois pour les méthodes et les fonctions qui prennent des fonctions en paramètre ou qui renvoient une fonction.

Exemple notable : La fonction map, disponible dans les collections

```
1 val salaries = Seq(20_000, 70_000, 40_000)
2 val doubleSalary = (x: Int) => x * 2
3 val newSalaries = salaries.map(doubleSalary) // List(40000, 140000, 80000)
```

scala

Ici nous avons défini une fonction `doubleSalary` dont nous l'utiliserons dans une `map` qui est HoF. L'exemple ici va doubler le salaire de chaque élément de la séquence.



Higher Order Function

Il est possible de factoriser le code :

```
1 val salaries = Seq(20_000, 70_000, 40_000)
2 val newSalaries = salaries.map(x => x * 2) // List(40000, 140000, 80000)
```

scala

Ou bien encore plus (niveau flemmard)

```
1 val salaries = Seq(20_000, 70_000, 40_000)
2 val newSalaries = salaries.map(_ * 2)
```

scala

Objectif principale des HoF : **Réduire considérablement les ligne de code**



Exemple sans HoF

Supposons que vous souhaitiez disposer de méthodes permettant d'augmenter le salaire d'une personne en fonction de différents facteurs.

Sans créer de fonction d'ordre supérieur, cela pourrait ressembler à ceci :

```
1  object SalaryRaiser { scala
2      def smallPromotion(salaries: List[Double]): List[Double] =
3          salaries.map(salary => salary * 1.1)
4
5      def greatPromotion(salaries: List[Double]): List[Double] =
6          salaries.map(salary => salary * math.log(salary))
7
8      def hugePromotion(salaries: List[Double]): List[Double] =
9          salaries.map(salary => salary * salary)
10 }
```



Exemple avec HoF

Remarquez que les trois méthodes affichés est quasiment identique au coefficient près. Nous allons extraire le code dupliqué vers une HoF.

```
1  object SalaryRaiser { scala
2      private def promotion(salaries: List[Double], promotionFunction: Double
      => Double): List[Double] = salaries.map(promotionFunction)
3
4      def smallPromotion(salaries: List[Double]): List[Double] =
5          promotion(salaries, salary => salary * 1.1)
6      def greatPromotion(salaries: List[Double]): List[Double] =
7          promotion(salaries, salary => salary * math.log(salary))
8      def hugePromotion(salaries: List[Double]): List[Double] =
9          promotion(salaries, salary => salary * salary)
10 }
```



Exemple une HoF qui renvoi une valeur

```
1  def urlBuilder(ssl: Boolean, domainName: String): (String, String) => String = { scala
2    val schema = if (ssl) "https://" else "http://"
3    (endpoint: String, query: String) => s"$schema$domainName/$endpoint?$query"
4  }
5
6  val domainName = "www.example.com"
7  def getURL = urlBuilder(ssl=true, domainName)
8  val endpoint = "users"
9  val query = "id=1"
10 val url = getURL(endpoint, query) // "https://www.example.com/users?id=1": String
```



Les paramètres multiples de list (FoldLeft)

Attention : Egalement utilisé par les Data Engineers

Les méthodes peuvent admettre plusieurs paramètres, prenons un exemple pour l'illustrer :

```
1 def multiply(a: Int)(b: Int): Int = a * b
2
3 val result = multiply(3)(4) // Appel la fonction avec deux paramètres
  distincts
4 println(result) // Affiche 12
```

scala

Ici :

- La multiplication possède deux paramètre a et b
- Nous pouvons appeler `multiply` en donnant les deux paramètres en séquence.

```
multiply(3)(4)
```



Les paramètres multiples de list (FoldLeft)

Il est possible de faire une application partiel. C'est à dire que vous pouvez donner le reste des arguments plus tard

```
1  val multiplyByTwo = multiply(2)_ // Application partiel, on fixe 2  
   à l'avance, mais on remplit plus-tard  
2  
3  println(multiplyByTwo(5)) // On applique le reste, donc le résultat est 10  
   car 2*5
```

scala



Maintenant exemple plus complexe :

Nous allons mixer les HoF avec les MPL, car Scala peut inférer plus facilement les types lorsque vous utilisez un MPL, surtout les HoF.

```
1 trait Iterable[A]
2   ...
3   def foldLeft[B](z: B)(op: (B, A) => B): B
4   ...
```

scala

- Ici nous avons deux paramètre
 - z qui représente la valeur initiale
 - op qui est une fonction HoF prenant deux paramètre (B, A) et renvoie un B



Maintenant exemple plus complexe :

Supposons que notre opération est $(m, n) \Rightarrow m + n$

```
1 val number = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
2 val res = number.foldLeft(0)((m, n) => m + n)
```

scala

Ici l'opération est la somme de tout les éléments de number, dont le résultat est stocké à l'aide d'un accumulateur initialisé à 0.

Nous prenons l'accumulateur pour l'ajouter au prochain élément de la liste.

02

SBT (Scala build tool)





Introduction

Il existe plusieurs outils permettant de compiler un projet Scala.

- Ant
- Maven
- Graddle
- **SBT**

Nous utiliserons sbt qui est le plus utilisé de tous.



Architecture d'un dossier

```
1  build.sbt
2  project/
3  src/
4  -- main/
5    |-- java/
6    |-- resources/
7    |-- scala/
8  |-- test/
9    |-- java/
10   |-- resources/
11   |-- scala/
12 target/
```

Le build.sbt

Pour créer un projet Scala, il faut un fichier build.sbt et un fichier scala.

```
1 name := "HelloWorld"
2 version := "1.0"
3 scalaVersion := "2.13.15"
```

sbt

Pour lancer un programme de compilation et de run :

```
1 sbt run
```

- La première fois que vous lancez sbt, elle va télécharger tout les dépendances nécessaire pour fonctionner le projet.



Commandes SBT

On peut s'amuser à lancer plus rapidement les instance de compilations avec une persistance de sbt. Pour cela, nous utilisons tout d'abord

```
1 sbt
```

et ensuite

```
1 run
```

à chaque fois que vous souhaitez relancer le code + compilation.

Remarque : En cas de présence de plusieurs fonction Main, sbt vous demandera lequel utiliser (autrement dit, quel object à utiliser pour run).



Et pour ajouter une dépendance :

```
1 libraryDependencies += List(  
2   "org.scala-lang" % "scala-reflect" % scalaVersion.value,  
3   "org.scala-lang" % "scala-compiler" % scalaVersion.value,  
4 )
```

scala

La recherche des librairie se réalise sur le site officiel de Maven Repository :

<https://mvnrepository.com/>



Et enfin pour packager

Pour packager un code Scala via SBT, rien de plus simple :

```
1 sbt package
```

Place au Travaux Pratique 2 !